

Coding for Artificial Intelligence

Functions and Object References

Problem Set 6

Functions, Arguments, and Mutable vs. Immutable Data

Lab Project: The DNA Sequence Marker Finder

Course	Coding for AI (Week 6)
Author	Dr. Prateek, Assistant Professor, School of AI
Format	In-class practical, 1 hour 30 minutes total, inclusive of write-up and submission
Total Marks	100

Read this document in full before you begin writing code.

Contents

1	Objective	2
2	Task 1: Building search4letters (30 minutes)	2
2.1	Step-by-Step Execution	2
3	Task 2: Tracking Memory References (45 minutes)	3
3.1	The Core Demonstration	3
3.2	The Discovery Challenge	3
3.3	The Slice Experiment	4
3.4	Bonus Experiment: Mutation vs. Reassignment (Optional)	4
4	Edge Cases to Consider	5
5	Suggested Time Allocation	5
6	Deliverables	5
	Academic Integrity	5

1 Objective

In this practical you will act as a bio-informatician processing genetic data. Across two connected tasks, you will practice **writing parameterized functions** with positional, keyword, and default arguments, and then use those same functions to uncover one of the most important – and most frequently misunderstood – facts about Python: what actually happens to an object when it is passed into a function.

By the end of this practical you should be able to:

- (i) write and call a function using positional arguments, keyword arguments, and default parameter values interchangeably,
- (ii) use `id()` to determine whether two variables refer to the **same** object in memory or to two different ones, and
- (iii) explain precisely why mutating a list inside a function is visible outside it, while *reassigning* a parameter inside a function is not.

Time budget. This entire activity — coding, testing, and writing up your submission — is designed to fit in **90 minutes**. A suggested breakdown is given in Section 5.

2 Task 1: Building `search4letters` (30 minutes)

Open an editor and write the following parameterized function, which finds which target nucleotide markers are present in a DNA sequence:

```
1 def search4letters(phrase: str, letters: str = 'ATCG') -> set:
2     """Return the set of target 'letters' found in a supplied 'phrase'
3     ."""
4     # set(letters) creates a unique pool of target markers
5     # set(phrase) represents the genetic material to analyze
    return set(letters).intersection(set(phrase))
```

Reading the annotations. `phrase: str` and `letters: str = 'ATCG'` are **type hints**: they document that both parameters are expected to be strings, and that `letters` has a default value of `'ATCG'` if the caller does not supply one. `-> set` documents that the function returns a `set`. Type hints in Python are not enforced at runtime — they are purely documentation for the reader (and for tools like type checkers) — but they make a function’s contract explicit at a glance.

2.1 Step-by-Step Execution

Run each of the following calls and record the result:

1. **Positional arguments:** `search4letters('TGGACC', 'GC')`
2. **Keyword arguments:** `search4letters(letters='CG', phrase='TGGACC')`
3. **Defaults:** `search4letters('TGGACC')` — this should automatically search for the default `'ATCG'` markers, since no second argument is supplied.

Call	Result
<code>search4letters('TGGACC', 'GC')</code>	<code>{'G', 'C'}</code>
<code>search4letters(letters='CG', phrase='TGGACC')</code>	<code>{'G', 'C'}</code>
<code>search4letters('TGGACC')</code>	<code>{'A', 'T', 'C', 'G'}</code>

Why the first two calls match. Positional and keyword arguments are two different *syntax* for supplying the same values to the same parameters — they do not change what the function computes. Both calls above bind `phrase` to `'TGGACC'` and `letters` to `'CG'`; only the notation differs.

Note on set ordering. Python does not guarantee the order in which set elements print. If your console shows the same letters as the table above but in a different order, your answer is still correct — a set is defined by which elements it contains, not by any ordering among them.

3 Task 2: Tracking Memory References (45 minutes)

This task uses a second function, unrelated to Task 1, to investigate how Python actually passes arguments into functions.

3.1 The Core Demonstration

```

1 def analyze_sequence(sequence_list, marker):
2     print(f"Inside Function (Start) — ID: {id(sequence_list)} | Data: {sequence_list}")
3     # In-place modification (mutating the shared object)
4     sequence_list.append(marker)
5     print(f"Inside Function (End) — ID: {id(sequence_list)} | Data: {sequence_list}")
6
7 # Driver code
8 dna_database = ['AATCCG', 'TGGCTA']
9 print(f"Global Scope (Before) — ID: {id(dna_database)} | Data: {dna_database}")
10
11 analyze_sequence(dna_database, 'CGAT')
12
13 print(f"Global Scope (After) — ID: {id(dna_database)} | Data: {dna_database}")

```

Run this and confirm that all three printed ID values are identical, and that `dna_database` in the global scope has `'CGAT'` appended to it — **even though `analyze_sequence` never uses a return statement.**

Note on `id()` values. The specific number `id()` returns is a memory address in CPython and will be different every time you run the program, and different again on a different computer. What matters is not the literal number, but whether the **same** number appears in all three print statements (proving it is the same object throughout) or whether it **changes** (proving a new object was created). Do not expect your numbers to match this document's — only the pattern of sameness matters.

3.2 The Discovery Challenge

Before reading further, write a short answer to this question: **why was `dna_database` modified in the global scope, even though `analyze_sequence` never returned anything?**

Answer. Python's argument-passing model is often described imprecisely as either "pass by reference" (the C++ sense) or "pass by value" (the sense used for primitives in Java or C) — **neither term is accurate for Python.** The precise description

is **call by object reference** (sometimes called “call by sharing”): when you call `analyze_sequence(dna_database, 'CGAT')`, the parameter `sequence_list` inside the function is bound to the *exact same list object* that `dna_database` refers to in the global scope — not a copy of it, and not a reference-to-a-reference in the C++ sense either. Because `list.append()` is a **mutating** method — it changes the object in place, rather than creating a new one — and both names point at that one object, the change is visible through either name. No `return` statement is needed, because nothing new was ever created; the one existing object was simply edited.

3.3 The Slice Experiment

Now rewrite the function call to pass a **shallow copy** of the list, created with a full slice, instead of the original:

```

1 dna_database = ['AATCCG', 'TGGCTA']
2 print(f"Global Scope (Before)    - ID: {id(dna_database)} | Data: {
   dna_database}")
3
4 analyze_sequence(dna_database[:], 'CGAT')
5
6 print(f"Global Scope (After)     - ID: {id(dna_database)} | Data: {
   dna_database}")

```

Observe the IDs and the global variable’s contents. You should find that the ID printed *inside* the function no longer matches the ID of `dna_database` in the global scope, and that `dna_database` itself is **unchanged** after the call — still exactly `['AATCCG', 'TGGCTA']`, with no `'CGAT'` appended.

Why slicing breaks the connection. `dna_database[:]` builds a **new** list object containing copies of the same references found in the original — a *shallow* copy. “Shallow” matters here because the two strings inside the list (`'AATCCG'` and `'TGGCTA'`) are not themselves duplicated; only the outer list is a new object. Since strings are immutable, this distinction does not cause problems in this particular exercise, but it explains why the technique is called a *shallow* copy rather than a full, independent duplicate. The function receives this new list, appends to *it*, and the original `dna_database` — a completely separate object by this point — is left untouched.

3.4 Bonus Experiment: Mutation vs. Reassignment (Optional)

For a deeper understanding, try rewriting `analyze_sequence` to use reassignment instead of `.append()`:

```

1 def analyze_sequence_rebind(sequence_list, marker):
2     print(f"    Inside (Start) - ID: {id(sequence_list)}")
3     sequence_list = sequence_list + [marker]    # REBINDS the local
         name
4     print(f"    Inside (End)   - ID: {id(sequence_list)} | Data: {
       sequence_list}")

```

Call this with the **original** (unsliced) `dna_database` and observe that, this time, **even without slicing**, the global list is left unchanged. The ID printed at the start of the function matches the global ID, exactly as in Section 3 — but the ID printed at the end does *not*, because `sequence_list + [marker]` builds an entirely new list and `sequence_list = ...` then re-points the *local* name at it. The global `dna_database` was never touched, because nothing ever called a mutating method on the object it refers to; the local name was simply reassigned to

point somewhere else. This is the single clearest way to see the difference between **mutating** an object (visible everywhere that holds a reference to it) and **reassigning** a variable (visible only in the scope where the reassignment happened).

4 Edge Cases to Consider

1. **Calling `search4letters` with an empty phrase**, e.g., `search4letters('')`. Confirm this returns an empty set rather than raising an error — the intersection of any set with an empty set is always empty.
2. **Passing a mix of positional and keyword arguments in the same call**, e.g., `search4letters('TGGACC', letters='GC')`. Confirm this behaves identically to using keyword arguments for both, and note that a positional argument can never come *after* a keyword argument in the same call.
3. **Appending to a list passed as a slice more than once**. Confirm that repeated calls using `dna_database[:]` each create a fresh, independent copy, so no accumulated changes ever reach the global list, no matter how many times you call it this way.

5 Suggested Time Allocation

Phase	Description	Duration
Task 1	<code>search4letters</code> : positional, keyword, and default calls	30 minutes
Task 2	Mutation demo, Discovery Challenge, and the Slice Experiment	40 minutes
Testing	Edge cases from Section 4; optional Bonus Experiment	10 minutes
Documentation & Submission	Header comment, final write-up	10 minutes
Total		90 minutes

6 Deliverables

1. **Source Code:** A single file named `dna_marker_finder.py` containing both `search4letters` and `analyze_sequence`, together with the driver code demonstrating all required calls from Tasks 1 and 2 (including the slice-copy call).
2. **Header Documentation:** A comment block at the top of your file explaining, in your own words, the term *call by object reference* and why it is different from both “pass by value” and “pass by reference” as those terms are used in other languages.
3. **Submission Note:** Your written answer to the Discovery Challenge in Section 3.2 (100–150 words), written *before* reading the provided answer, followed by one sentence noting whether your prediction was correct.

Academic Integrity

You may discuss approaches with peers during the supervised practical session, but the code and documentation you submit must be written independently and must reflect your own understanding. Submissions found to be substantially copied will be treated in accordance with the institute’s academic integrity policy.